ONNXim: A Fast, Cycle-Level Multi-Core NPU Simulator

Hyungkyu Ham, Wonhyuk Yang, Yunseon Shin, Okkyun Woo, Guseul Heo, Sangyeop Lee, Jongse Park, *Senior Member, IEEE*, and Gwangsun Kim, *Member, IEEE*

***Abstract*—As DNNs (Deep Neural Networks) demand increasingly higher compute and memory requirements, designing efficient and performant NPUs (Neural Processing Units) has become more important. However, existing architectural NPU simulators lack support for high-speed simulation, multi-core modeling, multi-tenant scenarios, detailed DRAM/NoC modeling, and/or different deep learning frameworks. To address these limitations, this work proposes *ONNXim*, a fast cycle-level simulator for multi-core NPUs in DNN serving systems. For ease of simulation, it takes DNN models in the ONNX graph format generated from various deep learning frameworks. In addition, based on the observation that typical NPU cores process tensor tiles from SRAM with *deterministic* compute latency, we model computation accurately with an event-driven approach, avoiding the overhead of modeling cycle-level activities. ONNXim also preserves dependencies between compute and tile DMAs. Meanwhile, the DRAM and NoC are modeled in cycle-level to properly model contention among multiple cores that can execute different DNN models for multi-tenancy. Consequently, ONNXim is significantly faster than existing simulators (e.g., by up to 365× over Accel-sim) and enables various case studies, such as multi-tenant NPUs, that were previously impractical due to slow speed and/or lack of functionalities.**

***Index Terms*—DNN inference, multi-tenancy, NPU, ONNX, simulator.**

I. Introduction

As THE accuracy of DNNs rapidly improves, they are becoming more widely adopted across various domains while demanding more compute and memory resources. Thus, it has become very important to achieve high performance and efficiency with NPUs.

Since architectural design exploration relies heavily on simulators, various cycle-level NPU simulators have been proposed. However, some prior approaches [1], [2], [3] lack support for critical aspects of NPUs such as the modeling of multi-core NPUs and/or detailed modeling for important shared resources (e.g., DRAM). Detailed modeling of shared resources is especially important for server-class NPUs that require multi-tenancy for high resource utilization [4]. Other simulators [5], [6], [7], [8] do support multi-core NPUs and detailed DRAM modeling, but significantly sacrifice the simulation speed.[1] Given that both DNNs and NPUs are becoming larger, increasing the time complexity of the simulation, it is important to provide fast simulation speeds. While FPGA implementations [9] achieve high speed, they cannot be used to model server-class NPUs that do not fit into an FPGA. Many simulators also require model descriptions in special formats rather than standard formats. Thus, existing simulators do not meet all the requirements for exploring future NPU designs.

To overcome the limitations of existing simulators, we propose ONNXim, a fast, cycle-level NPU simulator that supports multi-core NPUs, multi-tenancy, and detailed modeling of shared resources. We leverage ONNX (Open Neural Network Exchange) [11], an open standard format widely used for converting DNNs for different frameworks, including TensorRT for NVIDIA GPUs [12]. By using ONNX graphs as the input format, the user can easily simulate different DNNs from various deep learning frameworks without manual conversion to a specialized model description format. We leverage the ONNX runtime to optimize the graph and Timeloop [3] to optimize the mapping of operations to NPU cores. Custom fuse kernels (e.g., FlashAttention [13]) are modeled by adding them to our template kernel library. ONNXim also supports extensible global scheduler policies, which are necessary for studying multi-tenant workloads.

ONNXim achieves high simulation speed based on the observation that typical NPU cores with systolic arrays (SAs) process tensor tiles from on-chip scratchpad memory with deterministic latency [2]. Since its compute latency can be determined by the sizes of tensor tiles and the SA, we avoid modeling all individual arithmetic operations in a fine-grained manner, unlike conventional CPU/GPU simulators, while still preserving simulation accuracy. Meanwhile, shared resources with contention are modeled in detail using cycle-accurate simulators as they introduce non-determinism. While prior work [8] adopts a similar approach, ONNXim improves upon it in three significant ways. First, ONNXim enables DNN inferences with dynamic input shapes, which is important for modeling the growing key-value cache in large language models (LLMs). Second, we enable a more diverse set of operations beyond GEMM and CONV, including layer normalization and skip connections, which can collectively take up a significant portion of runtime [14]. Third, in ONNXim, the generation and execution of the dynamic instruction sequence for cores are optimized for fast simulation speed.

Received 25 August 2024; accepted 13 October 2024. Date of publication 22 October 2024; date of current version 27 November 2024. This work was supported in part by IITP (RS-2021-II210871, 2019-0-01906, and IITP-2024-RS-2024-00437866) and NRF (RS-2023-00277080 and RS-2024-00415602), funded by the MSIT of Korea. *Hyungkyu Ham and Wonhyuk Yang contributed equally to this work. (Corresponding author: Gwangsun Kim.)*

Hyungkyu Ham, Yunseon Shin, Okkyun Woo, and Gwangsun Kim are with the Department of Computer Science and Engineering, POSTECH, Pohang 37673, South Korea (e-mail: hhk971@postech.ac.kr; ysshin@postech.ac.kr; okkyun.w@postech.ac.kr; g.kim@postech.ac.kr).

Wonhyuk Yang is with the Graduate School of Artificial Intelligence, POSTECH, Pohang 37673, South Korea (e-mail: wonhyuk@postech.ac.kr).

Guseul Heo, Sangyeop Lee, and Jongse Park are with the School of Computing, KAIST, Daejeon 34141, South Korea (e-mail: gsheo@casys.kaist.ac.kr; sangyeop-lee@casys.kaist.ac.kr; jspark@casys.kaist.ac.kr).

ONNXim is publicly available at https://github.com/PSAL-POSTECH/ONNXim

Digital Object Identifier 10.1109/LCA.2024.3484648

[1] We consider GPU simulators [5], [6], [7] in our discussion since GPUs are widely used to accelerate DNNs. Among them, we mainly use Accel-Sim [5] since it is faster than NaviSim [7] and NVArchSim [6] is not publicly available.

1556-6056 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Xiamen University. Downloaded on July 29,2026 at 06:21:54 UTC from IEEE Xplore. Restrictions apply.

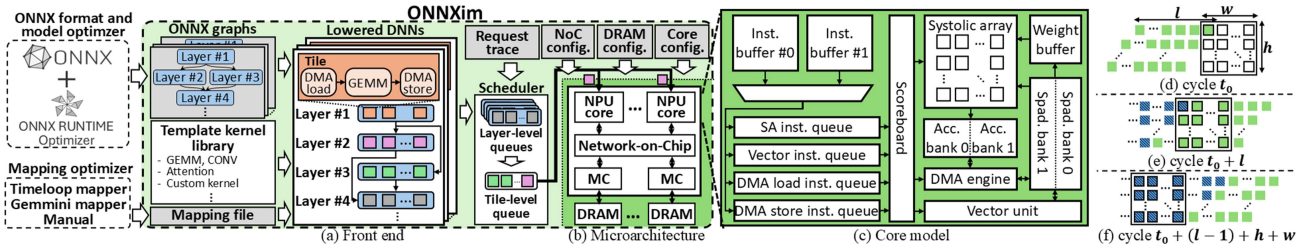


Fig. 1. Overview of ONNXim simulator (a–c) and SA operation timing model (d–f). (d) An SA instruction (with green input) begins execution. (e) Another SA instruction (with shaded blue input) can begin execution with overlap. (f) the first SA instruction (green) finishes.

As a result, ONNXim achieves simulation speedups of up to 365× over Accel-Sim [5] while maintaining high accuracy against the Gemmini RTL model [9] for DNN inference. We also demonstrate the usefulness of ONNXim through case studies on multi-tenant workloads and the impact of DNN architectures that were previously impractical due to slow speeds and/or lack of functionalities.

II. ONNXim

ONNXim is a cycle-level simulator for inference-oriented multi-core NPUs with SAs (Fig. 1). It is designed to enable high simulation speed for large DNN inference tasks while accurately modeling contention for shared resources. For multi-tenant scenarios, it supports multiple scheduling policies (e.g., time-sharing and spatial-sharing) and can be easily extended to support other policies. To enable the simulation of various models written in different frameworks (e.g., PyTorch and TensorFlow) without manual conversion to a custom format, ONNXim takes standard ONNX graphs as input for model description.

A. Front End

Model optimization and lowering: The input DNN can be first optimized using the ONNX runtime's offline graph optimizer to reduce computation and memory traffic. Its optimization techniques include constant folding, redundancy elimination, and fusion, such as fusing CONV/MHA (Multi-Head Attention) with batch/layer normalization layers and/or skip connection. Then, ONNXim lowers each node (or layer) in the ONNX graph to multiple tile operations using the tiled mapping scheme chosen by a mapping optimizer (e.g., Timeloop [3]) and the kernel templates in our template library. The library provides templates for basic layers, such as GEMM and CONV, as well as custom fused kernels (e.g., FlashAttention [13]) that users can choose from. Users can also add their own custom kernels written in our core ISA.

ISA: The kernel templates for tile operations use our NPU core's ISA to specify operations using the following instructions: 1) `MVIN/MVOUT` for DMA load/store between scratchpad memory and DRAM, 2) `GEMM Preload` for loading weights into the SA, 3) `GEMM` for performing matrix multiplication on the SA, 4) `IM2COL` for performing image-to-column transformations, and 5) vector operations (e.g., GELU). The ISA is an extension of the Gemmini ISA [9], with additional instructions for vector operations and activation functions.

Scheduler: To enable multi-tenant scenarios, ONNXim takes a request trace in JSON format that describes the inference request stream with various DNNs, batch sizes, and timestamps. The requests can be scheduled with different scheduling policies. A time-sharing policy schedules a layer from one request at a time before switching to a layer from another request in a round-robin fashion. Alternatively, a spatial-sharing policy partitions the NPU cores among multiple models, enabling them to execute concurrently. Users can also easily add new, sophisticated policies using our modular implementation.

In the scheduler (Fig. 1(a)), a *layer-level queue* exists for each inference request and tracks layer-wise dependency information from the ONNX graph. A layer without unresolved dependencies can then be popped from the queue and lowered into multiple tile-level operations that are pushed into a *tile-level queue*. Multiple independent layers can be pushed into the tile-level queue, allowing for concurrent execution. The tile-level queue exists per user-specified NPU core partition, such that an idle NPU core context can pop a tile-level operation and begin execution, based on the scheduling policy.

B. NPU Microarchitecture Model

Core Organization. ONNXim models a typical NPU core's organization with an SA, weight buffer, scratchpad memory, accumulator, and vector units (Fig. 1(c)). The accumulator includes its own SRAM as well as arithmetic units for accumulation operations. The core receives a tile operation from the global scheduler and executes its instructions from the instruction buffer. To support double-buffering, there are two instruction buffers and each PE in the SA also has two weights for the two contexts, adopting the Gemmini SA model [9]. The scratchpad and accumulator memory also have two banks and their interfaces match the width (w) and height (h) of the SA to avoid bottlenecks. An instruction scheduler issues instructions to the DMA engine or execution pipeline when there are no structural or data hazards, by tracking the unit's availability and the dependencies between compute and DMA instructions.

Core implementation: ONNXim improves the simulation speed for NPU core computation by adopting an event-driven approach and avoiding the cycle-by-cycle simulation of the SA and vector unit as in conventional CPU/GPU simulators. The SA can be implemented with different dataflows: weight-stationary, input-stationary, and output-stationary. Their relative performance depends on the dimensions of the SA and input tensor sizes [2]. We assume the widely-adopted weight-stationary SA [15] with dimensions $w \times h$. For an SA operation, the weights are first preloaded into the SA over h cycles. Then the SA is fed with an input matrix tile of size $l \times h$ in a skewed manner (Fig. 1(d)), taking $(l-1) + w + h$ cycles for the computation (Fig. 1(f)). During the SA operation, the SA operation for the next tile can be partially overlapped to minimize pipeline bubbles if the next input tile is ready in the scratchpad memory and the preloaded weight for the current SA operation can be reused. Specifically, the next tile can start the SA operation after the top row of the current tile is pushed to the SA (Fig. 1(e)). The SA events are generated deterministically based on this timing model in ONNXim. Thus, high SA utilization can be achieved for large matrix multiplications with a long steady-state operation. Vector instructions are also modeled to use

Authorized licensed use limited to: Xiamen University. Downloaded on July 29,2026 at 06:21:54 UTC from IEEE Xplore. Restrictions apply.

TABLE I
NPU CONFIGURATIONS AND DNNS USED FOR EVALUATION

NPU Configuration		
Parameter	**Mobile NPU**	**Server NPU**
SA size, # of cores	8×8, 4 @ 1 GHz	128×128, 4 @ 1 GHz
Vector unit (# of ALUs)	128	2048
Scratchpad (acc.) / core	64 KB (16 KB)	32 MB (4 MB)
DRAM	DDR4 (12 GB/s)	2 HBM2 (614 GB/s)

Evaluated DNNs	
DNN Model	**Description**
ResNet-18/50	(C:3, H:224, W:224) input image
GPT-3 Small	512-token prefill (P), 100-token generation (G)
BERT-large	Process a 512-token input prompt
Llama-3 (8B)	Generating a single token with a 1024-token context

operands from on-chip SRAM and, therefore, have deterministic latency calculated from the operand's size, vector unit width, and instruction latency configurations. In addition, ONNXim supports heterogeneous NPU cores with different SA and SRAM configurations to model NPUs designed to execute DNNs with different scales efficiently [8].

Shared resources: We model the shared off-chip DRAM using Ramulator 2.0 [16], a fast cycle-level simulator that can model various DRAM devices (e.g., DDR, HBM, and LPDDR), to accurately model memory traffic contention. For tensor data movement to/from the cores, the per-core DMA engine generates memory requests with the DRAM-access-granularity and sends them to memory controllers.

For NoCs, ONNXim supports a versatile cycle-accurate simulator, Booksim [17], as well as a simple latency and bandwidth model. While a detailed NoC model may not be necessary for NPUs with abundant on-chip bandwidth, multi-die NPUs [18] with limited die-to-die interconnect bandwidth require an accurate interconnect model. By integrating these cycle-level models, ONNXim can accurately model contention for shared resources.

III. EVALUATION

A. Methodology

We compared the simulation speed of ONNXim with that of existing simulators – mNPUsim [8], SMAUG [10], and Accel-Sim [5]. We did not include SCALE-Sim [2] and STONNE [1] in the comparison since they cannot model multi-core NPUs, which are essential for modeling server-class NPUs. ONNXim was evaluated in two versions: one with a simple latency-bandwidth NoC model (`ONNXim-SN`) and the other with the Booksim [17] NoC model (`ONNXim`). In both cases, DRAM was modeled with Ramulator 2.0 [16].

We used two NPU configurations, a `Mobile NPU` similar to Arm's Ethos-U55 [19] and a `Server NPU` similar to Google's TPUv4i [4], for evaluation (Table I). Because SMAUG did not work if the systolic array was larger than 8 × 8, we used it only for the `Mobile NPU` speed comparison. For comparisons with Accel-Sim, we used NPU-like GPU configurations: `Mobile NPU`-like GPU assumed a single GA102 SM at 566MHz (for similar FLOPS) and 128 KB L2 cache with a single GDDR6 channel with an 8-bit bus width; `Server NPU`-like GPU was assumed to have 75 GA102 SMs at 1.1GHz and 138 MB L2 cache with two HBM2 stacks. Note that, while these NPU-like *GPUs* have L2 caches, the *NPUs* we model do not have L2 caches.

For speed comparisons with existing NPU simulators, we focused on GEMM operations, as they can dominate the simulation time and some of the simulators did not support other operations such as layer normalization and skip connections. However, DNN models in Table I were also simulated end-to-end for speed comparison with Accel-Sim, as well as for validation and case studies. We used FlashAttention [13] for Transformer-based models. We used an AMD EPYC 7773X CPU for simulation speed evaluations. All simulators we used, including ONNXim, are single-threaded. For simulation runs that were shorter than two hours, we report the average of five measurements.

B. Simulation Speed

Compared to Accel-Sim, `ONNXim-SN` achieved significant simulation speedups of 79 × and 5.4× for `Server NPU` and `Mobile NPU`, respectively (Fig. 2(a)). While `ONNXim` provided a lower speedup due to the detailed NoC model, it was still substantially faster than the other simulators. Other simulators were overall even slower than Accel-Sim. The speedup was particularly pronounced for `Server NPU`, since as the size of the systolic array increases, a larger matrix tile can be processed at once on the simulated NPU core. In contrast, the number of dynamic instructions in the trace for Accel-Sim is proportional to the number of fixed-size tiles from the GEMM. It also simulates all low-level instructions at the warp level, including data movements using loads/stores by numerous warps through the cache hierarchy and register file as well as fine-grained Tensor Core instructions. ONNXim was also significantly faster than Accel-Sim in end-to-end DNN simulations (Fig. 2(b)). For the `Server NPU` running GPT-3(P) and ResNet-50, `ONNXim-SN` was 60–365× faster than Accel-Sim.

C. Validation With Core RTL Model

The accuracy of ONNXim's core model was validated against the RTL implementation of Gemmini [9]. While calculating the compute latency of a systolic array is straightforward, other aspects of the core's execution, including data movement between different SRAMs and the overlap of data movement with systolic array compute latency, should also be accurately modeled and validated. We measured only the core's execution time to isolate the randomness from memory and NoC latencies. Fig. 2(c) shows that ONNXim achieves high accuracy with a mean absolute error of 0.61% and a correlation of 0.99 for various workloads. Since Gemmini does not support softmax, normalization, and skip connections, these operations were excluded from the validation.

D. Case Study on a Multi-Tenant Workload

For a multi-tenant NPU, the contention for limited DRAM bandwidth can significantly impact its performance. As a case study, we examine the performance interference from ResNet-50 on the tail latency of GPT-3(G) on a `Server NPU`. The NPU cores were spatially partitioned such that core 0 executed GPT-3(G), while cores 1–3 executed ResNet-50. In Fig. 2(d), we plot the 95th percentile (p95) latency of TBT (Time-Between-Token) from a total of 500 tokens generated by GPT-3(G) co-executed with ResNet-50 inferences across different batch sizes. The TBT distribution of GPT-3(G) was significantly impacted by the ResNet-50 tasks, increasing the p95 latency by 47% as the batch size for ResNet-50 varied from 1 to 32. Using a larger batch for ResNet-50 increases the memory bandwidth demand, resulting in more contention with the memory traffic from GPT-3(G). Since inference tasks have latency constraints [4], request scheduling must be done carefully, and ONNXim can be used to study the impact of scheduling policies.

Authorized licensed use limited to: Xiamen University. Downloaded on July 29,2026 at 06:21:54 UTC from IEEE Xplore. Restrictions apply.

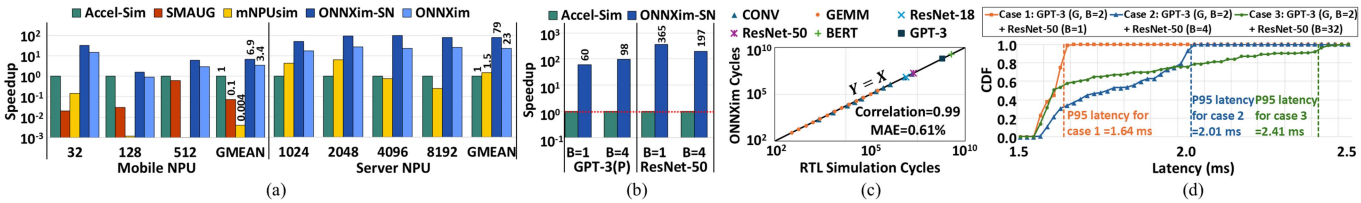


Fig. 2. Comparison of simulation speed, accuracy, and latency: (a) Simulation speeds over Accel-Sim for GEMM. (X-axis: size of each dimension N for $N \times N \times N$ GEMMs.); (b) Speedups for DNNs with varying batch sizes ('B') compared to Accel-Sim; (c) Validation of ONNXim against Gemmini RTL [8] using an 8×8 SA for GEMM, CONV, and various DNNs with a batch size of 1; (d) Distribution of TBT for GPT-3's generation phase when run with ResNet-50 on the same multi-core NPU with different batch sizes.

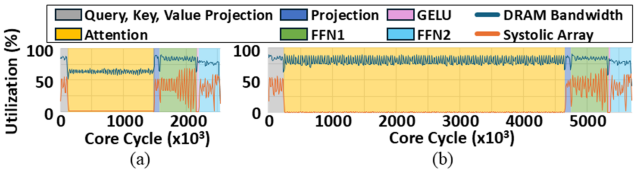


Fig. 3. Impact of different attention mechanisms on resource utilization.

E. Case Study on the Impact of Attention Mechanism

In the generation phase of Transformer-based LLMs, the attention mechanism can account for a significant portion of inference time. With the MHA from the original Transformer, the query vector of a newly generated token is multiplied by the key vector of all previous tokens, requiring a long GEMV operation. Since the GEMV operation is memory-bandwidth-bound, the NPU cores are significantly underutilized. To address this bottleneck, recent LLMs, such as Llama-3 [20], replace the MHA with Grouped Query Attention (GQA), where a group of attention heads shares the same key-value pairs, reducing the size of the GEMV operation.

We studied the impact of the attention mechanism choice on inference time and resource utilization in Fig. 3. Compared to the original Llama-3 with GQA, the modified Llama-3 with MHA substantially increases the latency of the attention mechanism, which is memory-bound and underutilizes the NPU core. The simulations took 7 and 14 minutes for the two cases on a single CPU core, which is acceptable for many studies, given the large size of the LLM with 8 billion parameters, a long context length of 1023 tokens, and a large batch size of 128. This result demonstrates that, thanks to the detailed DRAM model and fast simulation speed, ONNXim can properly model the key characteristics of LLMs and enable studies of LLM architecture on future NPU designs, and vice versa.

IV. Conclusion

In this work, we introduce ONNXim, a cycle-level multi-core NPU simulator that offers high simulation speed and detailed modeling of shared resources. By leveraging the industry-standard ONNX format for the simulator's DNN model inputs, ONNXim supports the simulation of existing DNNs implemented in various deep learning frameworks without manual conversion to a custom format. Based on the observation that the compute latency of a systolic array can be determined from its dimensions and the input tile sizes, ONNXim achieves high simulation speedups of up to 384× compared to prior cycle-level NPU simulators. Our case studies also demonstrate that ONNXim can be used to study various scenarios, including multi-tenant DNN inference and HW-aware LLM architecture designs.

References

[1] F. Muñoz-Martínez, J. L. Abellán, M. E. Acacio, and T. Krishna, "STONNE: Enabling cycle-level microarchitectural simulation for DNN inference accelerators," *IEEE Comput. Archit. Lett.*, vol. 20, no. 2, pp. 122–125, Jul./Dec. 2021.

[2] A. Samajdar, J. M. Joseph, Y. Zhu, P. Whatmough, M. Mattina, and T. Krishna, "A systematic methodology for characterizing scalability of DNN accelerators using SCALE-sim," in *Proc. IEEE Int. Symp. Perform. Anal. Syst. Softw.*, 2020, pp. 58–68.

[3] A. Parashar et al., "Timeloop: A systematic approach to DNN accelerator evaluation," in *Proc. IEEE Int. Symp. Perform. Anal. Syst. Softw.*, 2019, pp. 304–315.

[4] N. P. Jouppi et al., "Ten lessons from three generations shaped Google's TPUv4i: Industrial product," in *Proc. ACM/IEEE 48th Int. Symp. Comput. Archit*, 2021, pp. 1–14.

[5] M. Khairy, Z. Shen, T. M. Aamodt, and T. G. Rogers, "Accel-sim: An extensible simulation framework for validated GPU modeling," in *Proc. ACM/IEEE 47th Annu. Int. Symp. Comput. Archit.*, 2020, pp. 473–486.

[6] O. Villa et al., "Need for speed: Experiences building a trustworthy system-level GPU simulator," in *Proc. 2021 IEEE Int. Symp. High-Perform. Comput. Archit.*, 2021, pp. 868–880.

[7] Y. Bao et al., "NaviSim: A highly accurate GPU simulator for amd RDNA GPUs," in *Proc. Int. Conf. Parallel Archit. Compilation Techn.*, 2022, pp. 333–345.

[8] S. Hwang, S. Lee, J. Kim, H. Kim, and J. Huh, "mNPUsim: Evaluating the effect of sharing resources in multi-core NPUs," in *Proc. 2023 IEEE Int. Symp. Workload Characterization*, 2023, pp. 167–179.

[9] H. Genc et al., "Gemmini: Enabling systematic deep-learning architecture evaluation via full-stack integration," in *Proc. 58th ACM/IEEE Des. Automat. Conf.*, 2021, pp. 769–774.

[10] S. Xi et al., "Smaug: End-to-end full-stack simulation infrastructure for deep learning workloads," *ACM Trans. Archit. Code Optim.*, vol. 17, no. 4, pp. 1–26, 2020.

[11] Introduction to ONNX, 2020. [Online]. Available: https://onnx.ai/onnx/intro/

[12] NVIDIA, Nvidia tensorrt. (n.d). [Online]. Available: https://docs.nvidia.com/deeplearning/tensorrt/quick-start-guide/index.html

[13] T. Dao et al., "Flashattention: Fast and memory-efficient exact attention with IO-awareness," in *Proc. 36th Int. Conf. Neural Inf. Process. Syst.*, 2022, pp. 16 344–16 359.

[14] S. Ghodrati et al., "Tandem processor: Grappling with emerging operators in neural networks," in *Proc. 29th ACM Int. Conf. Architectural Support Program. Lang. Operating Syst.*, 2024, pp. 1165–1182.

[15] N. P. Jouppi et al., "In-datacenter performance analysis of a tensor processing unit," in *Proc. 44th Annu. Int. Symp. Comput. Archit.*, 2017, pp. 1–12.

[16] H. Luo, Y. C. Tuğrul, F. N. Bostancı, A. Olgun, A. G. Yağlıkçı, and O. Mutlu, "Ramulator 2.0: A modern, modular, and extensible DRAM simulator," *IEEE Comput. Archit. Lett.*, vol. 23, no. 1, pp. 112–116, Jan./Jun. 2024.

[17] N. Jiang et al., "A detailed and flexible cycle-accurate network-on-chip simulator," in *Proc. IEEE Int. Symp. Perform. Anal. Syst. Softw.*, 2013, pp. 86–96.

[18] Y. S. Shao et al., "Simba: Scaling deep-learning inference with multi-chip-module-based architecture," in *Proc. 52nd Annu. IEEE/ACM Int. Symp. Microarchit.*, 2019, pp. 14–27.

[19] ARM, Ethos-u55, 2020. [Online]. Available: https://www.arm.com/products/silicon-ip-cpu/ethos/ethos-u55

[20] Meta llama 3, 2024. [Online]. Available: https://llama.meta.com/llama3/

Authorized licensed use limited to: Xiamen University. Downloaded on July 29,2026 at 06:21:54 UTC from IEEE Xplore. Restrictions apply.